

Core Java Basics

1. What is Java? Why is it called platform-independent?

Java is a high-level, object-oriented, platform-independent language. Java source code compiles to bytecode, which runs on any system having a JVM, making it "write once, run anywhere."

2. Difference between JDK, JRE, and JVM?

JVM executes bytecode. JRE = JVM + libraries (runtime). JDK = JRE + development tools (compiler, debugger).

3. Is Java a pure object-oriented language?

No, because it uses primitive data types (int, char, float, etc.) which are not objects.

4. What is bytecode in Java?

Bytecode is the intermediate code generated by the Java compiler (.class file) that the JVM executes.

5. Difference between == and .equals()?

== compares references (memory locations). .equals() compares actual content/values.

6. What are wrapper classes?

Classes that wrap primitive types into objects (Integer for int, Double for double, etc.). Used in collections and for autoboxing/unboxing.

7. What is autoboxing and unboxing?

Autoboxing: automatic conversion of primitive to wrapper object. Unboxing: wrapper object to primitive.

8. What is the default value of local variables?

Local variables don't have default values—they must be initialized before use.

OOP Concepts

9. What are the four pillars of OOP?

Encapsulation, Inheritance, Polymorphism, Abstraction.

10. Difference between abstraction and encapsulation?

Abstraction hides implementation details (what an object does). Encapsulation hides data using access modifiers (how it's done).

11. What is inheritance? Types of inheritance in Java?

Inheritance allows a class to inherit properties from another class. Java supports single, multilevel, and hierarchical inheritance. Multiple inheritance is supported only through interfaces.

12. Why doesn't Java support multiple inheritance?

To avoid the diamond problem—ambiguity when two parent classes have methods with the same signature.

13. What is polymorphism?

The ability of an object to take many forms. Two types: compile-time (method overloading) and runtime (method overriding).

14. Difference between method overloading and overriding?

Overloading: same method name, different parameters, in the same class (compile-time).

Overriding: same signature in parent-child relationship (runtime).

15. Can we override static methods?

No. Static methods belong to the class, not the instance—they can be hidden but not overridden.

16. Difference between abstract class and interface?

Abstract class can have concrete methods, constructors, and state. Interface (pre-Java 8) had only abstract methods; Java 8+ allows default and static methods. A class can extend one abstract class but implement multiple interfaces.

17. What is the use of the super keyword?

To access parent class methods, variables, or constructors from a child class.

18. What is the this keyword?

Refers to the current object instance—used to differentiate instance variables from local variables and to call other constructors.

19. What is constructor chaining?

Calling one constructor from another using this() (same class) or super() (parent class).

20. Can a constructor be private?

Yes, used in Singleton design pattern to prevent object creation outside the class.

Strings

21. Why are Strings immutable in Java?

For security, thread safety, string pool optimization, and hashCode caching.

22. Difference between String, StringBuilder, and StringBuffer?

String is immutable. StringBuffer is mutable and thread-safe (synchronized). StringBuilder is mutable but not thread-safe (faster).

23. What is the String constant pool?

A special memory area in the heap where string literals are stored. Reused to save memory.

24. String s1 = "hello"; String s2 = new String("hello"); — are they equal?

s1 == s2 returns false (different memory locations). s1.equals(s2) returns true (same content).

Collections Framework

25. What is the Collection framework?

A unified architecture for storing and manipulating groups of objects, including List, Set, Map, Queue interfaces and their implementations.

26. Difference between ArrayList and LinkedList?

ArrayList uses a dynamic array (fast random access, slow insertion in middle). LinkedList uses a doubly-linked list (fast insertion/deletion, slow random access).

27. Difference between ArrayList and Vector?

ArrayList is not synchronized (faster). Vector is synchronized (thread-safe but slower) and is considered legacy.

28. Difference between HashMap and Hashtable?

HashMap is not synchronized, allows one null key and multiple null values. Hashtable is synchronized, no null keys or values allowed.

29. How does HashMap work internally?

Uses an array of buckets with linked lists. Uses hashCode() to determine bucket and equals() to compare keys. In Java 8+, when a bucket exceeds 8 entries, it converts to a balanced tree.

30. Difference between HashSet and TreeSet?

HashSet is unordered and uses hashing (faster). TreeSet maintains sorted order using a Red-Black tree.

31. Difference between Comparable and Comparator?

Comparable defines natural ordering using compareTo() inside the class. Comparator defines custom ordering using compare() in a separate class.

32. Difference between fail-fast and fail-safe iterators?

Fail-fast throws ConcurrentModificationException if collection is modified during iteration (ArrayList, HashMap). Fail-safe works on a clone (CopyOnWriteArrayList, ConcurrentHashMap).

33. Difference between List, Set, and Map?

List allows duplicates, ordered. Set doesn't allow duplicates, generally unordered. Map stores key-value pairs with unique keys.

Exception Handling

34. Difference between checked and unchecked exceptions?

Checked exceptions are checked at compile-time (IOException, SQLException). Unchecked exceptions occur at runtime (NullPointerException, ArithmeticException).

35. Difference between throw and throws?

throw explicitly throws an exception inside a method. throws declares exceptions a method might throw in its signature.

36. Difference between final, finally, and finalize?

final is a keyword (constants, prevent inheritance/overriding). finally is a block that always executes. finalize() is a method called by garbage collector before destroying an object.

37. Can we have try without catch?

Yes, with a finally block. Try must be followed by either catch or finally (or both).

38. Will finally block always execute?

Yes, except when System.exit() is called, JVM crashes, or an infinite loop runs in try.

39. What is the difference between Error and Exception?

Errors are serious problems that applications shouldn't try to handle (OutOfMemoryError, StackOverflowError). Exceptions are conditions that applications might want to catch.

Multithreading

40. Difference between Thread class and Runnable interface?

Implementing Runnable is preferred since Java doesn't support multiple inheritance—you can still extend another class. Thread is a class that itself implements Runnable.

41. Difference between start() and run() methods?

start() creates a new thread and internally calls run(). Calling run() directly executes the code in the current thread, not a new one.

42. What is synchronization?

A mechanism to control access to shared resources by multiple threads, preventing data inconsistency. Achieved using the synchronized keyword.

43. What is a deadlock?

A situation where two or more threads are blocked forever, each waiting for the other to release a lock.

44. Difference between sleep() and wait()?

sleep() pauses the thread for specified time without releasing the lock. wait() releases the lock and waits until notify() or notifyAll() is called.

Java 8 Features

45. What are lambda expressions?

Anonymous functions enabling functional programming. Syntax: (parameters) -> expression.

Example: (a, b) -> a + b.

46. What is a functional interface?

An interface with exactly one abstract method (Runnable, Comparator, Callable). Annotated with @FunctionalInterface.

47. What is the Stream API?

A way to process collections in a functional style with operations like filter(), map(), reduce(), and collect(). Supports lazy evaluation and parallel processing.

48. Difference between map() and flatMap()?

map() transforms each element one-to-one. flatMap() transforms and flattens nested structures (one-to-many).

49. What are default methods in interfaces?

Methods in interfaces with a default implementation, introduced in Java 8 to allow adding new methods without breaking existing implementations.

Memory Management

50. What is garbage collection in Java?

An automatic process that reclaims memory occupied by unused objects. Prevents memory leaks. Triggered by JVM—developers can suggest it via System.gc() but cannot force it.